

DAA Practical 06

Name:-Mo Atif Azmi

Batch:-A4_B4_51

Date:-19-10-2025

Aim: Construction of OBST

Problem Statement: Smart Library Search Optimization

Task 1:

Scenario:

A university digital library system stores frequently accessed books using a binary search mechanism. The library admin wants to minimize the average search time for book lookups by arranging the book IDs optimally in a binary search tree. Each book ID has a probability of being searched successfully and an associated probability for unsuccessful searches (when a book ID does not exist between two keys). Your task is to determine the minimum expected cost of searching using an Optimal Binary Search Tree (OBST).

Code:-

```

daa2.py > optimal_bst
1  import sys
2  def optimal_bst(keys, p, q):
3      n = len(keys)
4      cost = [[0] * (n + 1) for _ in range(n + 1)]
5      root = [[0] * (n + 1) for _ in range(n + 1)]
6      for i in range(n):
7          cost[i][i] = p[i] + q[i] + q[i + 1]
8          root[i][i] = i
9      for length in range(2, n + 1):
10         for i in range(n - length + 1):
11             j = i + length - 1
12             cost[i][j] = sys.maxsize
13             total_prob = sum(p[i:j + 1]) + sum(q[i:j + 2])
14             for r in range(i, j + 1):
15                 t = cost[i][r - 1] + cost[r + 1][j] + total_prob
16                 if t < cost[i][j]:
17                     cost[i][j] = t
18                     root[i][j] = r
19     return cost[0][n - 1]
20 keys = [10, 20, 30, 40]
21 p = [0.1, 0.2, 0.4, 0.3]
22 q = [0.05, 0.1, 0.05, 0.05, 0.1]
23 min_cost = optimal_bst(keys, p, q)
24 print(f"Minimum expected cost of OBST: {min_cost:.4f}")
25

```

Output:-

```

Minimum expected cost of OBST: 2.5500
PS C:\Users\DT USER\.vscode> python -u "c:\Users\DT USER\.vscode\daa2.py"
Minimum expected cost of OBST: 2.5500
PS C:\Users\DT USER\.vscode>

```

Task02

[Courses](#) [Tutorials](#) [Problem](#)[Editorial](#)[Submissions](#)[Comments](#)

Optimal binary search tree

Difficulty: **Hard**Accuracy: **50.02%**Submissions: **11K+**Points: **8**

Given a sorted array **keys[0.. n-1]** of search keys and an array **freq[0.. n-1]** of frequency counts, where **freq[i]** is the number of searches to **keys[i]**. Construct a binary search tree of all keys such that the total cost of all the searches is as small as possible.

Let us first define the cost of a BST. The cost of a BST node is level of that node multiplied by its frequency. Level of root is 1.

Example 1:

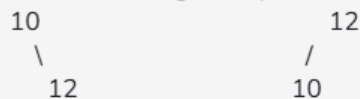
Input: $n = 2$

keys = {10, 12}

freq = {34, 50}

Output: 118**Explanation:**

There can be following two possible BSTs



The cost of tree I is $34*1 + 50*2 = 134$

The cost of tree II is $50*1 + 34*2 = 118$

Python3

Start Timer



```
1 class Solution:
2     def optimalSearchTree(self, keys, freq, n):
3         # dp[i][j] = minimum cost of BST that can be formed from keys[i..j]
4         dp = [[0 for _ in range(n) for _ in range(n)]
5
6         # sum_freq[i][j] stores sum of freq[i..j]
7         sum_freq = [[0]*n for _ in range(n)]
8         for i in range(n):
9             sum_freq[i][i] = freq[i]
10            for j in range(i+1, n):
11                sum_freq[i][j] = sum_freq[i][j-1] + freq[j]
12
13        # Fill dp for chains of length 1 to n
14        for length in range(1, n+1):
15            for i in range(n-length+1):
16                j = i + length - 1
17                dp[i][j] = float('inf')
18
19        # Try making each key in keys[i..j] the root
20        for r in range(i, j+1):
21            left = dp[i][r-1] if r > i else 0
22            right = dp[r+1][j] if r < j else 0
23            cost = left + right + sum_freq[i][j]
24            dp[i][j] = min(dp[i][j], cost)
25
26        return dp[0][n-1]
27
```

Compilation Completed

• Case 1

Input: 

```
2
10 12
34 50
```

Your Output:

```
118
```

Expected Output:

```
118
```